

Government College of Engineering, Jalgaon
(An Autonomous Institute of Government of Maharashtra)

Name :
Class : L. Y. B.Tech Computer
Subject : CO456U Cloud Computing Lab
Course Teacher: Prof. S. D. Cheke
Date of Performance :

PRN :
Batch:
Sem : VIIIth
Academic Year : 2025-26
Date of Completion :

Practical No. 4

Aim: Demonstrate database creation, data import, data insertion, select queries, joint, filtering and aggregations using AWS rds.

Requirements-

- AWS Account
- AWS RDS Service
- MySQL Workbench / CMD / Linux Terminal
- Basic SQL Knowledge
- Internet connectivity

Theory:

What is AWS RDS?

Amazon RDS (Relational Database Service) is a managed database service that supports databases like MySQL, PostgreSQL, SQL Server, MariaDB, and Oracle. RDS handles:

- Backups
- Patching
- Monitoring
- Scaling
- Security controls

SQL Operations Overview

Database Creation

A database is a structured collection of data.

Data import

Loading data into system

Data Insertion

INSERT command stores records in tables.

Joint

Combining data together

Select Queries

Fetching data from tables

Filtering

WHERE condition is used to filter data.

Aggregation

Functions like COUNT(), SUM(), AVG(), MAX(), MIN() are used to analyze data.

PRACTICAL STEPS

PART 1: Create Database in AWS RDS

Step 1: Open AWS Console

- Go to Amazon Web Services
- Login with your account

Step 2: Open RDS Service

- Search RDS
- Click RDS Dashboard

Step 3: Create Database

- Click Create Database

Select Options:

- Engine → MySQL
- Template → Free Tier

👉 Why?

- MySQL = relational DB
- Free tier = no cost for practical

Step 4: Set Credentials

- DB Name → mydb-instance
- Username → admin
- Password → password123

👉 This is used later to connect

Step 5: Connectivity Settings

- Public Access → Yes
- Port → 3306

👉 Why?

- Allows your laptop to connect to database

Step 6: Create Database

- Click Create Database
- Wait until status = Available

PART 2: Connect to Database

Step 7: Copy Endpoint

- Go to DB → Copy Endpoint

Step 8: Connect using MySQL

`mysql -h endpoint -u admin -p`

👉 Replace endpoint with actual value

Explanation:

- `mysql` → starts MySQL client
- `-h` → host (server address)
- `-u` → username
- `-p` → asks password

PART 3: Database Creation (Line-by-Line)

`CREATE DATABASE company_db;`

Explanation:

- `CREATE DATABASE` → creates new database
- `company_db` → database name

`USE company_db;`

Explanation:

- `USE` → selects database to work on

PART 4: Table Creation (Line-by-Line)

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    department VARCHAR(50),  
    salary INT  
);
```

Explanation:

- `CREATE TABLE` → creates table
- `employees` → table name

Columns:

- `emp_id INT` → employee ID (integer)
- `PRIMARY KEY` → unique identifier
- `name VARCHAR(50)` → text up to 50 characters
- `department VARCHAR(50)` → department name

- salary INT → salary value

```
CREATE TABLE departments (
  dept_id INT PRIMARY KEY,
  dept_name VARCHAR(50)
);
```

Explanation:

- Second table for join operation

PART 5: Data Insertion (Line-by-Line)

```
INSERT INTO employees VALUES
(1, 'Amit', 'IT', 50000),
(2, 'Neha', 'HR', 40000),
(3, 'Rahul', 'IT', 60000),
(4, 'Sneha', 'Finance', 45000);
```

Explanation:

- INSERT INTO → insert data
- VALUES → actual data rows
- Each row = one employee

```
INSERT INTO departments VALUE
(1, 'IT'),
(2, 'HR'),
(3, 'Finance');
```

PART 6: Data Import

```
LOAD DATA INFILE '/path/file.csv'
INTO TABLE employees
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Explanation:

- LOAD DATA INFILE → import CSV file
- /path/file.csv → file location
- FIELDS TERMINATED BY ',' → columns separated by comma
- LINES TERMINATED BY '\n' → new line = new record
- IGNORE 1 ROWS → skip header

PART 7: SELECT Queries

```
SELECT * FROM employees;
```

Explanation:

- SELECT * → fetch all columns
- FROM employees → table name

```
SELECT name, salary FROM employees;
```

Explanation:

- Select only specific columns

PART 8: Filtering (WHERE)

SELECT * FROM employees WHERE department = 'IT';

Explanation:

- WHERE → condition
- Shows only IT employees

SELECT * FROM employees WHERE salary > 45000;

PART 9: JOIN

SELECT e.name, d.dept_name

FROM employees e

JOIN departments d

ON e.department = d.dept_name;

Explanation:

- e → alias for employees
- d → alias for departments
- JOIN → combine two tables
- ON → condition for matching

👉 Output: Employee name + department name

PART 10: Aggregations

SELECT COUNT(*) FROM employees;

Explanation:

- Counts total employees

SELECT department, AVG(salary)

FROM employees

GROUP BY department;

Explanation:

- AVG → average salary
- GROUP BY → group data by department

SELECT MAX(salary) FROM employees;

SELECT MIN(salary) FROM employees;

Output:-

Aurora and RDS

> Dashboard

Aurora and RDS

<

Dashboard

Databases

Performance insights

Snapshots

Exports in Amazon S3

Automated backups

Reserved instances

Proxies

Subnet groups

Parameter groups

Option groups

Custom engine versions

Welcome to Aurora and RDS

Info

Amazon Relational Database Service (Amazon RDS) and Amazon Aurora are fully managed database services that simplify setup, operation, and scaling of relational databases in the cloud.



Create with express configuration in seconds
Create and query an Aurora PostgreSQL serverless database with pre-configured settings to get started quickly.

Create



Create with full configuration
Create a database with custom configurations and enable features you want across a variety of engines including Aurora (compatible with MySQL & PostgreSQL), MySQL, PostgreSQL, MariaDB, Oracle, Microsoft SQL Server, and IBM Db2. Or use an existing database using restore from S3.

Create

Restore from S3

Aurora and RDS

> Databases

> Create database

Create database

Info

Free plan has access to limited features and resources

The free plan limits the features and resources that are available for RDS and Aurora databases. Upgrade your account plan to remove all limitations. [Learn more](#)

Upgrade plan

Engine options

Engine type

Info

☐ Aurora (MySQL Compatible)



☐ Aurora (PostgreSQL Compatible)



☒ MySQL



☐ PostgreSQL



☐ MariaDB



☐ Oracle



☐ Microsoft SQL Server



☐ IBM Db2



Settings

Edition

☒ MySQL Community

Engine version

View the engine versions that support the following database features.

▼ Hide filters

☒ Show only versions that support the Multi-AZ DB cluster
Create a Multi-AZ DB cluster with one primary DB instance and two readable standby DB instances. Multi-AZ DB clusters provide up to 2x faster transaction commit latency and automatic failover in typically under 35 seconds.

☒ Show only versions that support the Amazon RDS Optimized Writes
Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

Engine version

MySQL 8.4.8

☐ Enable RDS Extended Support
Amazon RDS Extended Support is a paid offering. By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the [RDS for MySQL documentation](#).

DB instance identifier

Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

database-1

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

database-1

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Master username [Info](#)

Type a login ID for the master user of your DB instance.

admin

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management

You can use AWS Secrets Manager or manage your master user credentials.

☐ Managed in AWS Secrets Manager - *most secure*

RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ Self managed

Create your own password or have RDS create a password that you manage.

☐ Auto generate password

Amazon RDS can generate a password for you, or you can specify your own password.

Master password [Info](#)

Password strength [Average](#)

Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / ' * @

Confirm master password [Info](#)

[Aurora and RDS](#) > [Databases](#) > Create database



Additional credentials settings

Database authentication options [Info](#)

☒ Password authentication

Authenticates using database passwords.

☐ Password and IAM database authentication

Authenticates using the database password and user credentials through AWS IAM users and roles.

☐ Password and Kerberos authentication

Choose a directory in which you want to allow authorized users to authenticate with this DB instance using Kerberos Authentication.

Instance configuration

The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class [Info](#)

Hide filters

☐ Show instance classes that support Amazon RDS Optimized Writes [Info](#)

Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

☐ Include previous generation classes

☐ Standard classes (includes m classes)

☐ Memory optimized classes (includes r and x classes)

☒ Burstable classes (includes t classes)

Storage

Storage type [Info](#)

Provisioned IOPS SSD (io2) storage volumes are now available.

General Purpose SSD (gp2)

Baseline performance determined by volume size

Allocated storage [Info](#)

20

GiB

Allocated storage value must be 20 GiB to 6,144 GiB

Additional storage configuration

Connectivity [Info](#)



Compute resource

Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

☒ Don't connect to an EC2 compute resource

Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.

☐ Connect to an EC2 compute resource

Set up a connection to an EC2 compute resource for this database.

Virtual private cloud (VPC) [Info](#)

Choose the VPC. The VPC defines the virtual networking environment for this DB instance.

Default VPC (vpc-03645b51ef2858aee)

6 Subnets, 6 Availability Zones

After a database is created, you can't change its VPC. Only VPCs with a corresponding DB subnet group are listed.

After a database is created, you can't change its VPC.

DB subnet group [Info](#)

Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.

default

Public access [Info](#)

☐ Yes

RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.

☒ No

RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.

VPC security group (firewall) [Info](#)

Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

☒ Choose existing

Choose existing VPC security groups

☐ Create new

Create new VPC security group

Existing VPC security groups

Choose one or more options

default [×](#)

Availability Zone [Info](#)

No preference

RDS Proxy

RDS Proxy is a fully managed, highly available database proxy that improves application scalability, resiliency, and security.

☐ Create an RDS Proxy [Info](#)

RDS automatically creates an IAM role and a Secrets Manager secret for the proxy. RDS Proxy has additional costs. For more information, see [Amazon RDS Proxy pricing](#).

Certificate authority - optional [Info](#)

Using a server certificate provides an extra layer of security by validating that the connection is being made to an Amazon database. It does so by checking the server certificate that is automatically installed on all databases that you provision.

rds-ca-rsa2048-g1 (default)

Expiry: May 26, 2061

If you don't select a certificate authority, RDS chooses one for you.

✔ Successfully created database [database-10](#)

[View connection details](#)



You can use settings from database-10 to simplify configuration of suggested database add-ons while we finish creating your DB for you.

Databases (1)

Group resources



Modify

Actions

Create database

Filter by databases

< 1 >

DB identifier	Status	Role	Engine	Upgrade rollout order	Region ...	Size
database-10	✔ Available	Instance	MySQL Co...	SECOND	ap-southe...	db.t4g.micro


```
Command Prompt - mysql -h x + v
-> );
Query OK, 0 rows affected (0.19 sec)

mysql> INSERT INTO employees VALUES
-> (1, 'Amit', 'IT', 50000);
-> (2, 'Neha', 'HR', 40000);
-> (3, 'Rahul', 'IT', 60000);
-> (4, 'Sneha', 'Finance', 45000);
Query OK, 4 rows affected (0.18 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO departments VALUES
-> (1, 'IT');
-> (2, 'HR');
-> (3, 'Finance');
Query OK, 3 rows affected (0.24 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM employees;
+-----+-----+-----+-----+
| emp_id | name | department | salary |
+-----+-----+-----+-----+
| 1 | Amit | IT | 50000 |
| 2 | Neha | HR | 40000 |
| 3 | Rahul | IT | 60000 |
| 4 | Sneha | Finance | 45000 |
+-----+-----+-----+-----+
4 rows in set (0.16 sec)

mysql> SELECT name, salary FROM employees;
+-----+-----+
| name | salary |
+-----+-----+
| Amit | 50000 |
| Neha | 40000 |
| Rahul | 60000 |
| Sneha | 45000 |
+-----+-----+
4 rows in set (0.26 sec)

mysql> SELECT * FROM employees WHERE department = 'IT';
```

```
Command Prompt - mysql -h x + v

mysql> SELECT * FROM employees WHERE department = 'IT';
+-----+-----+-----+-----+
| emp_id | name | department | salary |
+-----+-----+-----+-----+
| 1 | Amit | IT | 50000 |
| 3 | Rahul | IT | 60000 |
+-----+-----+-----+-----+
2 rows in set (0.25 sec)

mysql> SELECT * FROM employees WHERE salary > 45000;
+-----+-----+-----+-----+
| emp_id | name | department | salary |
+-----+-----+-----+-----+
| 1 | Amit | IT | 50000 |
| 3 | Rahul | IT | 60000 |
+-----+-----+-----+-----+
2 rows in set (0.26 sec)

mysql> SELECT e.name, d.dept_name
-> FROM employees e
-> JOIN departments d
-> ON e.department = d.dept_name;
+-----+-----+
| name | dept_name |
+-----+-----+
| Amit | IT |
| Neha | HR |
| Rahul | IT |
| Sneha | Finance |
+-----+-----+
4 rows in set (0.19 sec)

mysql> SELECT COUNT(*) FROM employees;
+-----+
| COUNT(*) |
+-----+
| 4 |
+-----+
1 row in set (0.24 sec)
```

Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.

Mark-up and share

```
Command Prompt - mysql -h X + v

mysql> SELECT department, AVG(salary)
-> FROM employees
-> GROUP BY department;
+-----+-----+
| department | AVG(salary) |
+-----+-----+
| IT         | 55000.0000  |
| HR         | 40000.0000  |
| Finance    | 45000.0000  |
+-----+-----+
3 rows in set (0.19 sec)

mysql> SELECT MAX(salary) FROM employees;
+-----+
| MAX(salary) |
+-----+
| 60000       |
+-----+
1 row in set (0.19 sec)

mysql> SELECT MIN(salary) FROM employees;
+-----+
| MIN(salary) |
+-----+
| 40000       |
+-----+
1 row in set (0.20 sec)

mysql> SELECT department, AVG(salary)
-> FROM employees
-> GROUP BY department
-> HAVING AVG(salary) > 45000;
+-----+-----+
| department | AVG(salary) |
+-----+-----+
| IT         | 55000.0000  |
+-----+-----+
1 row in set (0.21 sec)

mysql> |
```



Conclusion:

In this practical, an AWS RDS MySQL instance was created and connected using the MySQL client. A database, table, and multiple records were created. Various SQL operations like select, filtering, and aggregations were successfully executed.

Prof. S. D. Cheke
Course Teacher